

Implémentation d'un CNN en C et CUDA pour la reconnaissance de chiffres manuscrits

Travail de maturité

Nathan Gasser

11 juillet 2026

Résumé

Les réseaux de neurones à convolution (CNN) sont très utilisés de nos jours pour la reconnaissance d'images. Nous allons en implémenter un en utilisant les langages C et CUDA. Son but sera de lire un chiffre manuscrit et sera entraîné sur le dataset MNIST, en utilisant une carte graphique NVIDIA.

Table des matières

1	Introduction	3
2	Modèle de reconnaissance d'images	3
3	MLP	3
3.1	Perceptron	3
3.2	Fonctions d'activation	4
3.2.1	La fonction <i>ReLU</i>	4
3.2.2	La fonction <i>tanh</i>	5
3.3	Plusieurs neurones	5
4	Backpropagation	6
4.1	Erreur	6
4.2	Backpropagation	7
4.2.1	SGD	8
5	CNN	8
5.1	Convolution	8
5.2	Pooling	8
5.3	CNN complet	8
6	Implémentation en C et CUDA	8
6.1	Moteur d'inférence en C	8
6.2	Moteur d'entraînement en CUDA	8
6.3	Interface utilisateur	8

1 Introduction

Pour la reconnaissance de chiffres manuscrits, il y a plusieurs méthodes possibles. Une des méthodes les plus simples est le perceptron à plusieurs couches (MLP), mais les CNN sont plus efficaces pour les tâches de reconnaissance d'images. Un CNN est un type de réseau de neurones qui a pour particularité d'utiliser des opérations de convolution pour extraire des caractéristiques de l'image. Ce TM aura pour but d'implémenter ce type de réseau de neurones en CUDA pour la partie entraînement et par la suite en C pour la partie inférence.

2 Modèle de reconnaissance d'images

Pour faire un modèle qui est capable de lire un chiffre manuscrit, il faut d'abord comprendre sous quelle forme donner l'image du chiffre manuscrit et également sous quelle forme se présentera la sortie du modèle.

Chaque chiffre manuscrit est une image de 28×28 pixels avec à chaque fois 256 différentes intensités de noir. C'est à dire que si la valeur du pixel est de 0, il correspond à un pixel blanc et si la valeur du pixel est de 255, il correspond à un pixel noir. Les valeurs entre deux correspondent à différentes intensités de gris. Notez que la valeur maximale d'un pixel correspond à 255 et non 256, tout simplement parce qu'il y a 256 valeurs différentes et que la valeur 0 est incluse, ce qui nous laisse 255 valeurs positives. Nous allons ensuite les diviser tous par 255, ce qui nous permet d'avoir des valeurs entre 0 et 1.0 que nous allons pouvoir donner au modèle. Nous avons donc $28 \times 28 = 784$ pixels dans une image. Notre modèle aura donc 784 entrées avec des valeurs comprises entre 0 et 1.0 à chaque fois.

Nous voulons obtenir en sortie 10 probabilités pour chacun des 10 chiffres possibles (de 0 à 9). Il y aura donc 10 neurones en sortie, chacun des différents chiffres sera assigné à un neurone et les valeurs de sortie des neurones se situeront entre -1.0 et 1.0 . Une valeur de -1.0 correspond à une certitude qu'il ne s'agit pas du chiffre associé à l'image et 1.0 correspond à une certitude qu'il s'agit du bon chiffre.

3 MLP

L'architecture la plus simple qu'on peut utiliser est le perceptron à plusieurs couches (MLP), il est composé de plusieurs couches composées elles même de plusieurs perceptrons. Bien qu'elle soit ancienne, elle est toujours utilisée elle même à l'intérieur de beaucoup d'autres architectures comme le CNN.

3.1 Perceptron

Le perceptron est la brique de base de l'intelligence artificielle. Il accepte un nombre fixe d'entrées (inputs) (N) et nous donne un seul nombre en sortie (output). Le perceptron peut en lui même déjà être utilisé pour créer des systèmes de classification simples.

Chaque entrée s'appelle x_n et est représentée sous forme d'un nombre naturel. Pour chaque entrée, il y a un poids (weight) associé (w_n). En plus des entrées et des poids, il y a un biais (bias) (b) associé à chaque neurone. Par convention, ces poids et biais seront appelés paramètres. Ces paramètres sont initialisés de manière aléatoire. Pour calculer la valeur de sortie, il faut également choisir une fonction d'activation (f). Il peut s'agir théoriquement d'à peu près n'importe quelle fonction mathématique, mais dans le cadre de ce TM, nous allons utiliser deux fonctions principalement : ReLU et tanh

Pour calculer la valeur de sortie de notre perceptron, il faudra d'abord calculer la valeur Z (z) en faisant une somme pondérée des entrées et du biais. Ensuite, nous pourrons appliquer la fonction d'activation à la valeur Z . La formule est la suivante :

$$y = f(z) = f\left(\sum_{n=1}^N w_n x_n + b\right)$$

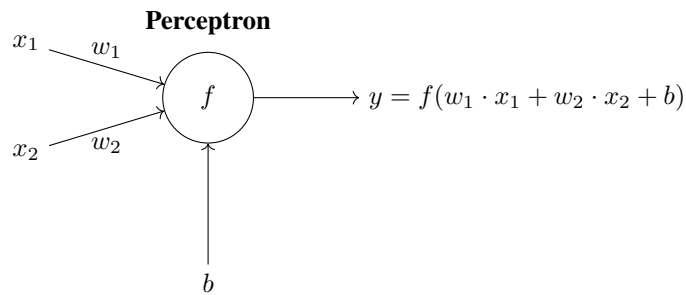


FIGURE 1 – Schéma d'un perceptron avec deux entrées (x_1 et x_2), deux poids (w_1 et w_2), un biais (b) et une fonction d'activation (f) qui produit la sortie (y).

3.2 Fonctions d'activation

La fonction d'activation permet d'ajouter de la non linéarité au modèle. Cela permet au modèle d'apprendre des motifs bien plus complexes. Sans fonction d'activation, le modèle ne pourrait apprendre que des motifs linéaires, ce qui est beaucoup trop limité pour des tâches de reconnaissance d'images.

3.2.1 La fonction *ReLU*

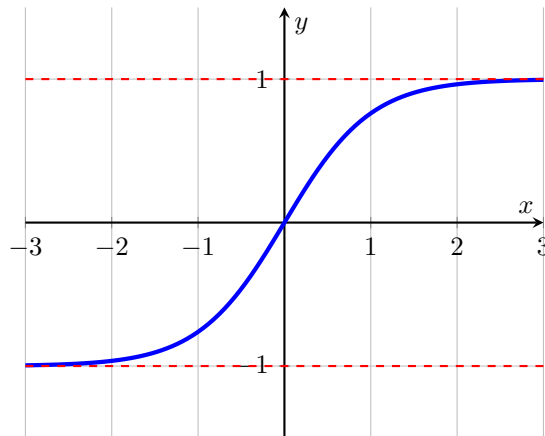
La fonction *ReLU* est la suivante :

$$ReLU(x) = \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases}$$

3.2.2 La fonction $\tanh$

La fonction $\tanh$ (tangente hyperbolique) est utilisée pour confiner les valeurs de sorties entre la valeur -1.0 et 1.0 . Dans le contexte de la reconnaissance d'images, elle permet de représenter le niveau de certitude de notre modèle par rapport à différentes prédictions. La fonction softmax est également souvent utilisée mais par simplicité j'ai décidé d'utiliser $\tanh$ pour ce TM.

Graphe de $\tanh(x)$



3.3 Plusieurs neurones

Pour former un MLP complet, il faut agencer plusieurs couches de plusieurs perceptrons ensemble. A part pour la couche initiale du modèle, la valeur de l'entrée numéro i dans le neurone numéro n dans la couche numéro c correspondra à la valeur de sortie du neurone $n - 1$ sur la couche $c - 1$. Si nous sommes sur la couche initiale du réseau, les entrées des différents neurones correspondent aux entrées du modèle (dans notre cas, les différents pixels d'une image). Pour les couches initiales et finales, il s'agit des couches d'entrées et de sorties respectivement.

Nous pouvons schématiser un MLP ainsi :

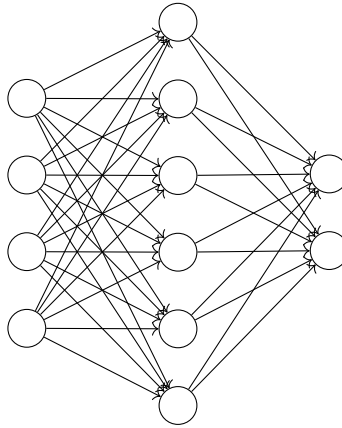


FIGURE 2 – Schéma d'un MLP avec 4 entrées et 2 sorties

4 Backpropagation

Pour que notre modèle puisse correctement prédire les sorties correctes, il va falloir l'entraîner. Sans entraînement, les prédictions seront tout simplement aléatoires. Pour entraîner notre modèle, nous allons utiliser la backpropagation qui est considérée comme l'algorithme le plus important dans le domaine du deep learning. Le but sera de faire la dérivée de la perte par rapport aux différents paramètres du modèle pour savoir comment modifier ces derniers tel que la perte diminue et que le modèle fasse donc de meilleures prédictions.

4.1 Erreur

Avec la backpropagation, notre but sera de minimiser la perte (loss) (L) du modèle. Si avec une certaine entrée, on obtient pour la sortie numéro n une valeur de y_n , alors que nous voulions obtenir une valeur de l_n , nous pouvons calculer la perte en soustrayant la valeur que nous attendions à la valeur que nous avons obtenu et élever le tout au carré. Cet algorithme s'appelle l'erreur quadratique (Mean Squared Error ou MSE en anglais). Nous pouvons ensuite additionner les pertes de toutes les sorties pour obtenir la perte totale du modèle.

$$L = (y_n - l_n)^2$$

La perte en elle même n'est intéressante que pour suivre l'évolution de la qualité du modèle. Comme expliqué plus haut, ce qui va vraiment nous intéresser, c'est la dérivée de la perte par rapport aux différents paramètres du modèle. C'est ici qu'entre en jeu la backpropagation. Pour calculer la dérivée de la perte par rapport aux différents poids et biais, nous allons utiliser la règle de la chaîne (chain rule). Pour commencer la chaîne, nous allons devoir calculer la dérivée des pertes par rapport aux différentes

sorties associées du modèle. Pour une sortie y_n associée à une valeur attendue l_n , la dérivée de la perte totale L par rapport à cette sortie est la suivante :

$$\frac{\partial L}{\partial y_n} = 2 \cdot (y_n - l_n)$$

4.2 Backpropagation

Ensuite, nous allons pouvoir utiliser la règle de la chaîne pour faire la backpropagation et calculer la dérivée de la perte par rapport aux différents paramètres du modèle. Pour commencer, il va falloir calculer la dérivée de la perte par rapport aux valeurs Z . Pour un neurone n dont la valeur de sortie équivaut à y_n , la dérivée de la perte par rapport à la valeur Z est la suivante si la fonction d'activation est *tanh* :

$$\frac{\partial L}{\partial z_n} = \frac{\partial L}{\partial y_n} \cdot \frac{\partial y_n}{\partial z_n} = \frac{\partial L}{\partial y_n} \cdot (1 - y_n^2)$$

et si la fonction d'activation est *ReLU* :

$$\frac{\partial L}{\partial z_n} = \frac{\partial L}{\partial y_n} \cdot \frac{\partial y_n}{\partial z_n} = \begin{cases} 0 & \text{si } z_n < 0 \\ \frac{\partial L}{\partial y_n} & \text{si } z_n \geq 0 \end{cases}$$

Ensuite, nous allons faire la dérivée à travers la valeur Z pour avoir la dérivée de la perte par rapport aux poids et aux biais. Pour un poids w_{wn} associé à une entrée i_{in} , la dérivée de la perte par rapport à ce poids est la suivante :

$$\frac{\partial L}{\partial w_{wn}} = \frac{\partial L}{\partial z_n} \cdot \frac{\partial z_n}{\partial w_{wn}} = \frac{\partial L}{\partial z_n} \cdot i_{in}$$

Et pour un biais b_n associé à un neurone n , la dérivée de la perte par rapport à ce biais est la suivante :

$$\frac{\partial L}{\partial b_n} = \frac{\partial L}{\partial z_n} \cdot \frac{\partial z_n}{\partial b_n} = \frac{\partial L}{\partial z_n}$$

Notez également la présence des valeurs y qui correspondent aux sorties des neurones. Pour un neurone dans la couche finale du modèle, nous avons déjà montré comment cette dérivée se calcule. Pour un neurone dans une autre couche, il va falloir faire la somme pondérée des dérivées des neurones de la couche suivante. Pour un neurone n de la couche c et M neurones dans la couche $c + 1$, la dérivée de la perte par rapport à la valeur de sortie de ce neurone est la suivante :

$$\frac{\partial L}{\partial y_{c,n}} = \sum_{m=1}^M \frac{\partial L}{\partial z_{c+1,m}} \cdot w_{c+1,m,n}$$

Les dérivées des neurones des couches précédant la couche de sortie nécessitant de connaître les dérivées des neurones de la couche qui suivent ces neurones, il est donc nécessaire de faire la backpropagation en calculant d'abord les dérivées des neurones de la couche de sortie, puis celles de la couche précédente et ainsi de suite jusqu'à la couche initiale.

4.2.1 SGD

Maintenant que nous avons les dérivées de la perte par rapport aux différents paramètres du modèle, nous allons devoir mettre à jour ces paramètres tel que la perte diminue. Pour ce faire, nous allons utiliser l'algorithme de descente de gradient stochastique (Stochastic Gradient Descent ou SGD en anglais). Cet algorithme est très simple à comprendre. Si nous connaissons la dérivée de la perte par rapport à un paramètre p_{ni} , nous savons que si nous changeons la valeur de ce paramètre dans le sens opposé à la dérivée, la perte va diminuer. Etant donné que nous ne voulions pas changer la valeur de ce paramètre trop rapidement, car nous pourrions ainsi dépasser le minimum de la fonction de perte, nous allons multiplier la dérivée par un scalaire appelé le taux d'apprentissage (learning rate) (α), dans le cadre de ce TM, sa valeur est de 0.05. La formule pour déterminer le nouveau paramètre (p_{nf}) est la suivante :

$$p_{nf} = p_{ni} - \alpha \cdot \frac{\partial L}{\partial p_{ni}}$$

5 CNN

5.1 Convolution

5.2 Pooling

5.3 CNN complet

6 Implémentation en C et CUDA

6.1 Moteur d'inférence en C

6.2 Moteur d'entraînement en CUDA

6.3 Interface utilisateur